MP7: Vanilla File System

Phani Jyothi Kurada
UIN: 335006950
CSCE611: Operating System

Assigned Tasks

- **Main:** Basic file system - **Completed**
- **Bonus Option 1:** Design of an extension to the basic file system to allow for files that are up to 64kB long - **Completed**
- **Bonus Option 2:** Implementation of the extensions proposed in Option 1 - **Completed**

System Design

This machine problem implements a minimal, single-level file system using two primary components: the `FileSystem` class, which manages global disk structures, and the `File` class, which provides sequential access to individual files. The system is built on top of the provided `SimpleDisk` device and supports only flat file naming; each file is uniquely identified by an integer `file_id`, with no directory hierarchy or path-based lookup.

The objective of this design is to showcase the basic mechanisms required for file allocation, metadata storage, and sequential I/O, without the complexity of a full-featured UNIX-like file system. As such, the implementation focuses on correctness, simplicity, and the clear separation of metadata management (handled by `FileSystem`) from per-file access operations (handled by `File`).

The objective of this design is to showcase the basic mechanisms required for file allocation, metadata storage, and sequential I/O, without the complexity of a full-featured UNIX-like file system. As such, the implementation focuses on correctness, simplicity, and the clear separation of metadata management (handled by `FileSystem`) from per-file access operations (handled by `File`).

FileSystem Class

The `FileSystem` class is responsible for file creation, deletion, lookup, and the overall management of disk-resident metadata. Internally, it maintains:

- An **inode table**, stored in block 0, which holds metadata entries for up to 64 files.
- A **free-block bitmap**, stored in block 1, used to record which disk blocks are currently allocated or free.
- A fixed offset `DATA_BLOCK_START`, ensuring that metadata blocks are separated from the region where file data is stored.

During formatting, the file system initializes both the inode table and the free-block bitmap and writes them to disk. When the system is mounted, these structures are read back into memory. Creating a new file proceeds as follows:

1. Check whether a file with the given identifier already exists.
2. Locate a free inode entry and a free data block.
3. Mark the selected data block as allocated in the bitmap.

4. Update and persist the modified inode table and free-block bitmap.

Deletion follows the inverse process: the file's data block is released back to the free list, and its inode entry is reset to indicate that it is no longer in use.

File Class

The `File` class provides the interface for sequential access to an individual file. It maintains:

- A reference to the file's `inode`.
- A 512-byte in-memory `cache` used for buffering the contents of the file's data block.
- A `current_position` variable that tracks progress during reads and writes.

All I/O operations operate directly on the cache. When a file is opened, its data block is loaded into the cache; when it is closed, any modifications are flushed back to disk. Reads copy up to `_n` bytes from the cache starting at the current position, while writes copy from the user buffer into the cache and may increase the file's length if necessary.

Inode Structure

Each `Inode` stores:

- `id` – File identifier.
- `length` – Current file length.
- `direct_block` – The disk block where the file data is stored.
- `valid` – Whether the inode is in use.

Only one data block is supported per file in the base version, enforcing a 512B file size limit. The inode structure is small enough to fit all 64 entries in one disk block.

Supported Operations

The following public methods in the `FileSystem` class are implemented:

- `CreateFile()` – Allocate an inode and block for a new file.
- `DeleteFile()` – Release all resources of the file.
- `Mount()` – Load inode table and block bitmap from disk.
- `Format()` – Write empty inode table and free bitmap to disk.
- `LookupFile()` – Retrieve inode reference given `file_id`.

The `File` class implements:

- `Read()`, `Write()` – For sequential file access.
- `Reset()`, `EoF()` – Manage reading position.

Code Description

Files Modified: `file.H`, `file.C`, `file_system.H`, `file_system.C`

file_system.H: Inode Class

The `Inode` class encapsulates the metadata required to manage individual files within the file system. It stores fields such as the file's unique identifier, its current length in bytes, and the block number that contains the file's data. In the base implementation, each inode maintains a single `block_number` corresponding to exactly one data block.

For the bonus extension, this design is expanded to support indirect addressing while preserving backward compatibility. The inode structure introduces an additional pointer for an indirect block, allowing the file to reference multiple data blocks when larger file sizes are enabled. Each inode also maintains a pointer to the parent `FileSystem` instance, enabling convenient access to utility routines such as block allocation and metadata persistence.

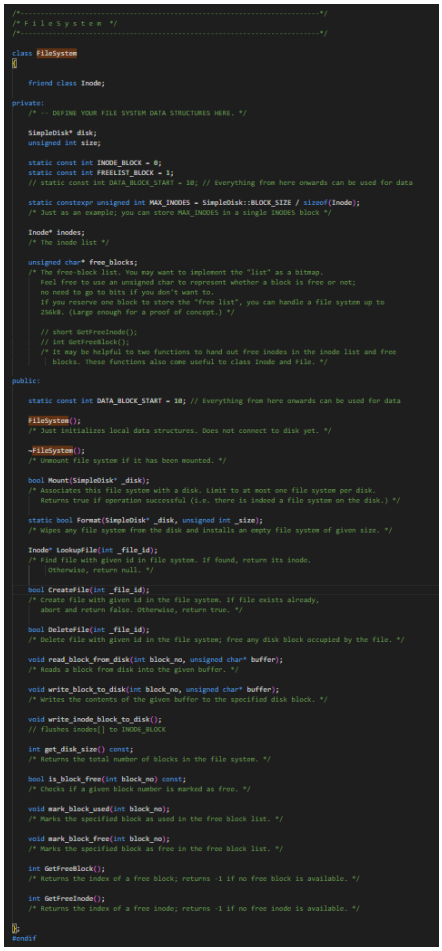
```
class Inode
{
    friend class FileSystem; // The inode is in an uncomfortable position between
    friend class File;       // File System and File. We give both full access
    // to the Inode.

private:
    long id; // File "name"

    /* You will need additional information in the inode, such as allocation
       information. */
    unsigned int length;       // File size in bytes
    int indirect_block;         // Block number containing up to 128 data block pointers
    bool valid;                // Is this inode in use?

    FileSystem* fs; // It may be handy to have a pointer to the File system.
    // For example when you need a new block or when you want
    // to load or save the inode list. (Depends on your
    // implementation.)

/* You may need a few additional functions to help read and store the
   inodes from and to disk. */
public:
    Inode() : id(-1), length(0), indirect_block(-1), valid(false), fs(nullptr) {}

    void SaveToDisk(int index);
    void LoadFromDisk(int index);
};
```

Figure 1: Structure of `Inode` class

file_system.H: FileSystem Class

The `FileSystem` class maintains all global state associated with the file system, including a pointer to the underlying disk device, the array of inodes, and the free-block bitmap. It also defines important constants such as `DATA_BLOCK_START`, which specifies the boundary between metadata blocks and the region where actual file data is stored.

The class provides core management routines such as `Mount()`, `Format()`, `CreateFile()`, and `DeleteFile()`, each responsible for initializing or updating persistent structures on disk. To support clean and modular design, the implementation additionally exposes helper functions for reading and writing disk blocks and for ensuring inode-table consistency. Utility functions such as `GetFreeInode()` and `GetFreeBlock()` simplify allocation logic and are reused across both `file.C` and `file_system.C`.



```
/*--------------------------------------------------------------------------*/
/* F i l e S y s t e m */
/*--------------------------------------------------------------------------*/

class FileSystem
{

    friend class Inode;

private:
    /* -- DEFINE YOUR FILE SYSTEM DATA STRUCTURES HERE. */

    SimpleDisk* disk;
    unsigned int size;

    static const int INODE_BLOCK = 0;
    static const int FREELIST_BLOCK = 1;
    // static const int DATA_BLOCK_START = 10; // Everything from here onwards can be used for data

    static constexpr unsigned int MAX_INODES = SimpleDisk::BLOCK_SIZE / sizeof(Inode);
    /* Just as an example; you can store MAX_INODES in a single INODES block */

    Inode* inodes;
    /* The inode list */

    unsigned char* free_blocks;
    /* The free-block list. You may want to implement the "list" as a bitmap.
       Feel free to use an unsigned char to represent whether a block is free or not;
       no need to go to bits if you don't want to.
       If you reserve one block to store the "free list", you can handle a file system up to
       256kB. (Large enough for a proof of concept.) */

       // short GetFreeInode();
       // int GetFreeBlock();
       /* It may be helpful to two functions to hand out free inodes in the inode list and free
          blocks. These functions also come useful to class Inode and File. */

public:

    static const int DATA_BLOCK_START = 10; // Everything from here onwards can be used for data

    FileSystem();
    /* Just initializes local data structures. Does not connect to disk yet. */

    ~FileSystem();
    /* Unmount file system if it has been mounted. */

    bool Mount(SimpleDisk* _disk);
    /* Associates this file system with a disk. Limit to at most one file system per disk.
       Returns true if operation successful (i.e. there is indeed a file system on the disk.) */

    static bool Format(SimpleDisk* _disk, unsigned int _size);
    /* Wipes any file system from the disk and installs an empty file system of given size. */

    Inode* LookupFile(int _file_id);
    /* Find file with given id in file system. If found, return its inode.
       Otherwise, return null. */

    bool CreateFile(int _file_id);
    /* Create file with given id in the file system. If file exists already,
       abort and return false. Otherwise, return true. */

    bool DeleteFile(int _file_id);
    /* Delete file with given id in the file system; free any disk block occupied by the file. */

    void read_block_from_disk(int block_no, unsigned char* buffer);
    /* Reads a block from disk into the given buffer. */

    void write_block_to_disk(int block_no, unsigned char* buffer);
    /* Writes the contents of the given buffer to the specified disk block. */

    void write_inode_block_to_disk();
    // Flushes inodes[] to INODE_BLOCK

    int get_disk_size() const;
    /* Returns the total number of blocks in the file system. */

    bool is_block_free(int block_no) const;
    /* Checks if a given block number is marked as free. */

    void mark_block_used(int block_no);
    /* Marks the specified block as used in the free block list. */

    void mark_block_free(int block_no);
    /* Marks the specified block as free in the free block list. */

    int GetFreeBlock();
    /* Returns the index of a free block; returns -1 if no free block is available. */

    int GetFreeInode();
    /* Returns the index of a free inode; returns -1 if no free inode is available. */

};
#endif
```

Figure 2: Structure of `FileSystem` class

file_system.C: Constructor and Destructor

The `FileSystem` constructor initializes the class's internal structures, including setting the inode array, free-block bitmap, and disk pointer to `nullptr`. These members are later populated when the file system is formatted or mounted.

The destructor ensures that all in-memory metadata is safely written back to disk before the object is destroyed. In particular, it flushes the inode table and free-block bitmap to their designated disk blocks to maintain consistency across executions. Any dynamically allocated memory is also released to avoid resource leaks.

```
/*--------------------------------------------------------------------------*/
/* CLASS FileSystem */
/*--------------------------------------------------------------------------*/

/*--------------------------------------------------------------------------*/
/* CONSTRUCTOR */
/*--------------------------------------------------------------------------*/

FileSystem::FileSystem() {
    Console::puts("In file system constructor.\n");
    inodes = nullptr;
    free_blocks = nullptr;
    disk = nullptr;
    size = 0;  // this will be set in Format or Mount
}

FileSystem::~FileSystem() {
    Console::puts("unmounting file system\n");
    /* Make sure that the inode list and the free list are saved. */
    write_inode_block_to_disk();
    disk->write(FREELIST_BLOCK, free_blocks);

    delete[] inodes;
    delete[] free_blocks;
}
```

Figure 3: Implementation of FileSystem `CONSTRUCTOR/DESTRUCTOR`

file_system.C: Mount()

The `Mount()` method attaches a `SimpleDisk` instance to the file system and reconstructs all necessary in-memory structures. It begins by storing the disk pointer and determining the total number of available blocks. The method then allocates space for the inode table and loads the on-disk inode block (block 0) into memory. Each inode entry is reconstructed from this block and assigned a back-pointer to the current `FileSystem` object.

Next, the method allocates memory for the free-block bitmap and reads its contents from block 1. Once both the inode table and free-block list are restored, the file system is fully initialized and ready for operation.

```
bool FileSystem::Mount(SimpleDisk * _disk) {
    Console::puts("Mounting file system from disk...\n");
    /* Here you read the inode list and the free list into memory */

    // Save pointer to the disk
    disk = _disk;
    this->size = disk->NaiveSize() / SimpleDisk::BLOCK_SIZE;

    // Allocate memory for inodes
    inodes = new Inode[MAX_INODES];

    // Load inode block (block 0) from disk
    unsigned char inode_block[SimpleDisk::BLOCK_SIZE];
    disk->read(INODE_BLOCK, inode_block);

    for (int i = 0; i < MAX_INODES; i++) {
        inodes[i] = ((Inode*)inode_block)[i];
        inodes[i].fs = this;  // set back pointer to FileSystem
    }

    // Allocate memory for free block list
    free_blocks = new unsigned char[SimpleDisk::BLOCK_SIZE];
    disk->read(FREELIST_BLOCK, free_blocks);

    Console::puts("Mount successful.\n");
    return true;
}
```

Figure 4: Implementation of FileSystem `Mount()`

file_system.C: Format()

The `Format()` method initializes the disk with a fresh file system layout. It is invoked prior to mounting and is responsible for constructing both the inode table and the free-block bitmap. The procedure begins by determining the total number of disk blocks and verifying that the disk is sufficiently large for the required metadata.

All disk blocks are then cleared by writing zeros throughout the device, ensuring that no residual data interferes with the new file system. A new inode array is created, with each inode initialized using the default constructor, and this array is written to block 0. The free-block bitmap is also constructed, marking all blocks as free except for block 0 (which stores the inode table) and block 1 (which stores the free-block list). These two metadata blocks are marked as allocated to prevent accidental reuse by file data.

```
bool FileSystem::Format(SimpleDisk * _disk, unsigned int _size) { // static!
    Console::puts("Formatting disk...\n");
    /* Here you populate the disk with an initialized (probably empty) inode list
       and a free list. Make sure that blocks used for the inodes and for the free list
       are marked as used, otherwise they may get overwritten. */

    // Step 1: Calculate how many blocks we need
    unsigned int total_blocks = _size / SimpleDisk::BLOCK_SIZE;

    if (total_blocks < 10) {
        Console::puts("Disk too small to format!\n");
        return false;
    }

    // Step 2: Zero out all blocks on disk
    unsigned char zero_block[SimpleDisk::BLOCK_SIZE] = {0};
    for (unsigned int i = 0; i < total_blocks; i++) {
        _disk->write(i, zero_block);
    }

    // Step 3: Write empty inode list to INODES block (block 0)
    Inode empty_inodes[MAX_INODES];
    for (int i = 0; i < MAX_INODES; i++) {
        empty_inodes[i] = Inode(); // default constructor sets valid = false
    }

    _disk->write(INODE_BLOCK, (unsigned char*)empty_inodes);

    // Step 4: Initialize free block list
    unsigned char free_list[SimpleDisk::BLOCK_SIZE] = {0};

    for (unsigned int i = 0; i < total_blocks; i++) {
        free_list[i] = 0; // 0 = free, 1 = used
    }

    // Mark INODE_BLOCK and FREELIST_BLOCK as used
    free_list[INODE_BLOCK] = 1;
    free_list[FREELIST_BLOCK] = 1;

    _disk->write(FREELIST_BLOCK, free_list);

    Console::puts("Disk formatted successfully.\n");
    return true;
}
```

Figure 5: Implementation of FileSystem `Format()`

file_system.C: LookupFile()
The `LookupFile()` method searches the inode table to locate a file associated with a specific identifier. It sequentially scans all inode entries and checks two conditions: (1) whether the inode is marked as valid, and (2) whether its `id` matches the requested `_file_id`. If a matching valid inode is found, the method returns a pointer to that inode. If no such entry exists after scanning the entire table, it returns `nullptr`. This lookup mechanism is essential for file creation, deletion, and access, as it provides a consistent way to map file identifiers to inode structures.

```
Inode * FileSystem::LookupFile(int _file_id) {
    Console::puts("looking up file with id = ");
    Console::puti(_file_id);
    Console::puts("\n");
    /* Here you go through the inode list to find the file. */

    for (int i = 0; i < MAX_INODES; ++i) {
        if (inodes[i].valid && inodes[i].id == _file_id) {
            return &inodes[i];
        }
    }
    return nullptr; // Not found
}
```

Figure 6: Implementation of FileSystem `LookupFile()`

file_system.C: CreateFile()
The `CreateFile()` method is responsible for allocating and initializing a new file within the file system. Its execution proceeds through the following steps:

- It first invokes `LookupFile()` to determine whether a file with the specified identifier already exists. If so, the operation is aborted.
- The method then scans the inode table for an unused (invalid) inode entry and consults the free-block bitmap to locate an available data block.
- Once both resources are identified, the selected data block is marked as allocated in the free-block list, and the updated bitmap is written back to disk.
- The chosen inode entry is populated with the file identifier, assigned data block, and appropriate metadata, and then persisted using `SaveToDisk()`.

By verifying all prerequisites before committing to any updates, the method ensures that file creation is performed safely and atomically.

```
bool FileSystem::CreateFile(int _file_id) {
    Console::puts("creating file with id:");
    Console::puti(_file_id);
    Console::puts("\n");

    // 1. Check if file already exists
    if (LookupFile(_file_id) != nullptr) {
        Console::puts("File already exists!\n");
        return false;
    }
    // 2. Find a free inode
    int inode_index = GetFreeInode();
    if (inode_index == -1) {
        Console::puts("No free inodes!\n");
        return false;
    }
    // 3. Find a free block for the indirect block
    int indirect_block_index = GetFreeBlock();
    if (indirect_block_index == -1) {
        Console::puts("No free blocks for indirect block!\n");
        return false;
    }
    // 4. Mark indirect block as used
    free_blocks[indirect_block_index] = 1;
    disk->write(FREELIST_BLOCK, free_blocks);
    // 5. Initialize the indirect block with 128 pointers set to -1
    unsigned char indirect_block_data[SimpleDisk::BLOCK_SIZE];
    for (int j = 0; j < 128; ++j) {
        ((int*)indirect_block_data)[j] = -1;
    }
    disk->write(indirect_block_index, indirect_block_data);
    // 6. Fill in inode
    Inode& inode = inodes[inode_index];
    inode.id = _file_id;
    inode.length = 0;
    inode.indirect_block = indirect_block_index;
    inode.valid = true;
    inode.fs = this;
    // 7. Save inode to disk
    inode.SaveToDisk(inode_index);

    Console::puts("File created successfully.\n");
    return true;
}
```

Figure 7: Implementation of FileSystem `CreateFile`

file_system.C: DeleteFile():
The `DeleteFile()` method removes a file with a given ID from the file system by:

- Iterating through the inode list to locate the file based on ID.
- If found, the corresponding data block (if valid and within range) is marked free in the free block list, and the updated list is written back to disk.
- The inode is cleared by resetting it to its default constructor state and then written back to disk using `SaveToDisk()`.

This ensures the file is fully removed from both metadata (inode) and data storage (block), restoring the associated disk block for future use.

```
bool FileSystem::DeleteFile(int _file_id) {
    Console::puts("deleting file with id:");
    Console::puti(_file_id);
    Console::puts("\n");
    /* First, check if the file exists. If not, throw an error.
       Then free all blocks that belong to the file and delete/invalidate
       (depending on your implementation of the inode list) the inode. */

    // 1. Find the inode
    for (int i = 0; i < MAX_INODES; ++i) {
        if (inodes[i].valid && inodes[i].id == _file_id) {
            // 2. Free all blocks used by the file
            // 2a. Read indirect block
            unsigned char indirect_data[SimpleDisk::BLOCK_SIZE];
            disk->read(inodes[i].indirect_block, indirect_data);
            // 2b. Free each allocated data block
            for (int j = 0; j < 128; ++j) {
                int data_block = ((int*)indirect_data)[j];
                if (data_block >= DATA_BLOCK_START && data_block < size) {
                    free_blocks[data_block] = 0;
                }
            }
            // 2c. Free the indirect block itself
            int indirect_block_num = inodes[i].indirect_block;
            if (indirect_block_num >= DATA_BLOCK_START && indirect_block_num < size) {
                free_blocks[indirect_block_num] = 0;
            }
            // 2d. Save updated freelist
            disk->write(FREELIST_BLOCK, free_blocks);

            // 3. Clear the inode
            inodes[i] = Inode(); // reset to default
            inodes[i].fs = this; // restore fs pointer
            inodes[i].SaveToDisk(i);

            Console::puts("File deleted successfully.\n");
            return true;
        }
    }
    Console::puts("File not found!\n");
    return false;
}
```

Figure 8: Implementation of FileSystem `DeleteFile()`

file_system.C: Disk I/O Utilities

The file system provides several helper functions to streamline block-level data transfers between memory and the underlying disk. The methods `read_block_from_disk()` and `write_block_to_disk()` act as lightweight wrappers around the corresponding `SimpleDisk` read and write operations, and are invoked frequently by both the `File` and `FileSystem` classes.

Additionally, the method `write_inode_block_to_disk()` serializes the in-memory inode array and writes it to block 0. This ensures that any changes resulting from file creation, deletion, or modification are consistently reflected on disk.

```
void FileSystem::read_block_from_disk(int block_no, unsigned char* buffer) {
    disk->read(block_no, buffer);
}

void FileSystem::write_block_to_disk(int block_no, unsigned char* buffer) {
    disk->write(block_no, buffer);
}

void FileSystem::write_inode_block_to_disk() {
    unsigned char inode_block[SimpleDisk::BLOCK_SIZE];
    for (int i = 0; i < MAX_INODES; ++i) {
        ((Inode*)inode_block)[i] = inodes[i];
    }
    disk->write(INODE_BLOCK, inode_block);
}
```

Figure 9: Helper methods for disk access in `FileSystem`

inode: SaveToDisk()
The `SaveToDisk()` method persists the current `Inode` instance to disk by updating the corresponding entry in the on-disk inode table stored in block 0. To accomplish this, the method first loads the entire inode block into memory, replaces the specific inode entry with the updated version, and then writes the modified block back to disk. This guarantees that any changes to the inode's metadata are reliably stored.

```
// Save this inode to disk at the appropriate index in the inode list block
void Inode::SaveToDisk(int index) {
    unsigned char buffer[SimpleDisk::BLOCK_SIZE];
    fs->disk->read(FileSystem::INODE_BLOCK, buffer);
    ((Inode*)buffer)[index] = *this;
    fs->disk->write(FileSystem::INODE_BLOCK, buffer);
}
```

Figure 10: Implementation of `Inode::SaveToDisk()`

inode: LoadFromDisk()
The `LoadFromDisk()` method retrieves a specific inode from disk based on its index within the inode table. It begins by reading the entire inode block from disk into memory. The inode entry at the specified index is then extracted, cast appropriately, and copied into the current `Inode` instance. This restores the inode's metadata exactly as it was stored on disk.

```
// Load this inode from disk into memory from index in the inode list block
void Inode::LoadFromDisk(int index) {
    unsigned char buffer[SimpleDisk::BLOCK_SIZE];
    fs->disk->read(FileSystem::INODE_BLOCK, buffer);
    *this = ((Inode*)buffer)[index];
}
```

Figure 11: Implementation of `Inode::LoadFromDisk()`

file.H & file.C: File Class Structure

These files define the primary structure and behavior of the `File` class. Each `File` object maintains references to the parent file system and its associated inode, along with an in-memory cache used to store the contents of the file's data block. The class also tracks the current read/write position within the file. The interface provides methods for reading and writing data, resetting the file pointer, checking for end-of-file conditions, and handling object construction and destruction.

```
class File  {

private:
    /* -- your file data structures here ... */

    /* You will need a reference to the inode, maybe even a reference to the
       file system.
       You may also want a current position, which indicates which position in
       the file you will read or write next. */

    FileSystem* fs;
    Inode* inode;
    int current_position;

    unsigned char block_cache[SimpleDisk::BLOCK_SIZE];
    /* It will be helpful to have a cached copy of the block that you are reading
       from and writing to. In the base submission, files have only one block, which
       you can cache here. You read the data from disk to cache whenever you open the
       file, and you write the data to disk whenever you close the file.
    */
    int GetBlockNumber(int logical_block_index, bool allocate_if_missing);


public:

    File(FileSystem * _fs, int _id);
    /* Constructor for the file handle. Set the 'current position' to be at the
       beginning of the file. */

    ~File();
    /* Closes the file. Deletes any data structures associated with the file handle. */

    int Read(unsigned int _n, char * _buf);
    /* Read _n characters from the file starting at the current position and
       copy them in _buf.  Return the number of characters read.
       Do not read beyond the end of the file. */

    int Write(unsigned int _n, const char * _buf);
    /* Write _n characters to the file starting at the current position. If the write
       extends over the end of the file, extend the length of the file until all data is
       written or until the maximum file size is reached. Do not write beyond the maximum
       length of the file.
       Return the number of characters written. */

    void Reset();
    /* Set the 'current position' to the beginning of the file. */

    bool EoF();
    /* Is the current position for the file at the end of the file? */

};

#endif
```

Figure 12: Structure of `File` class

file.C: File Constructor and Destructor

The constructor of the `File` class retrieves the corresponding inode from the file system and loads the file's data block into the in-memory cache, preparing it for sequential access. The destructor ensures

that any modifications made to the cached data are written back to disk and that the inode metadata is updated accordingly. This guarantees that file contents and metadata remain consistent across file opens and closes.

```
/*--------------------------------------------------------------------------*/
/* CONSTRUCTOR/DESTRUCTOR */
/*--------------------------------------------------------------------------*/

File::File(FileSystem *_fs, int _id) {
    Console::puts("Opening file.\n");

    fs = _fs;                          // Store FileSystem pointer
    inode = fs->LookupFile(_id);      // Lookup the inode
    assert(inode != nullptr);         // File must exist

    current_position = 0;
}

File::~File() {
    Console::puts("Closing file.\n");

    // Save updated inode metadata (e.g., file length)
    fs->write_inode_block_to_disk();
}
```

Figure 13: Implementation of `File Constructor and Destructor`

file.C: Read()

The `Read()` method retrieves data starting from the current file position, reading up to `_n` bytes or stopping earlier if the end of the file is reached. All data is sourced directly from the in-memory cache of the file's data block. After the requested bytes are copied into the user-provided buffer, the current file position is advanced to reflect the number of bytes successfully read.

```
int File::Read(unsigned int _n, char *_buf) {
    Console::puts("reading from file\n");

    int bytes_read = 0;
    while (bytes_read < (int)_n && !EoF()) {
        int logical_block = current_position / SimpleDisk::BLOCK_SIZE;
        int offset = current_position % SimpleDisk::BLOCK_SIZE;

        int block_num = GetBlockNumber(logical_block, false);
        if (block_num == -1) break;

        unsigned char temp_block[SimpleDisk::BLOCK_SIZE];
        fs->read_block_from_disk(block_num, temp_block);

        _buf[bytes_read] = temp_block[offset];
        current_position++;
        bytes_read++;
    }
    Console::puts("Finished reading file.\n");
    return bytes_read;
}
```

Figure 14: Implementation of `File::Read()`

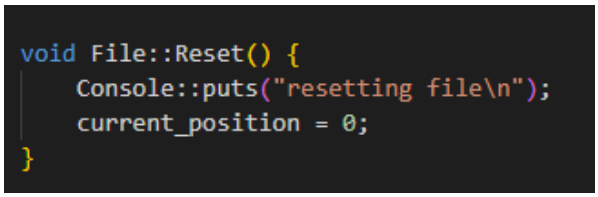
file.C: Write()

The `Write()` method stores up to `_n` bytes into the file beginning at the current position. All data is written directly into the in-memory cache for the file's data block. If the write operation extends beyond the file's previous logical length, the inode's `length` field is updated to reflect the new end of the file. The current position is advanced by the number of bytes successfully written.

```
int File::Write(unsigned int _n, const char *_buf) {
    Console::puts("writing to file\n");

    int bytes_written = 0;
    while (bytes_written < (int)_n && current_position < 128 * SimpleDisk::BLOCK_SIZE) {
        int logical_block = current_position / SimpleDisk::BLOCK_SIZE;
        int offset = current_position % SimpleDisk::BLOCK_SIZE;

        int block_num = GetBlockNumber(logical_block, true);
        if (block_num == -1) break;

        unsigned char temp_block[SimpleDisk::BLOCK_SIZE];
        fs->read_block_from_disk(block_num, temp_block);

        temp_block[offset] = _buf[bytes_written];
        fs->write_block_to_disk(block_num, temp_block);

        current_position++;
        bytes_written++;
    }

    if (current_position > (int)inode->length) {
        inode->length = current_position;
    }
    Console::puts("Finished writing to file.\n");
    return bytes_written;
}
```

Figure 15: Implementation of `File::Write()`

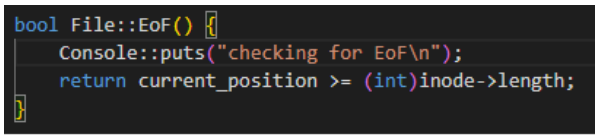
file.C: Reset(): Resets the file pointer to the beginning of the file.



```
void File::Reset() {
    Console::puts("resetting file\n");
    current_position = 0;
}
```

Figure 16: Implementation of `File::Reset()`

file.C: EoF(): Returns whether the file pointer has reached or passed the end of file, based on the inode length.



```
bool File::EoF() {
    Console::puts("checking for EoF\n");
    return current_position >= (int)inode->length;
}
```

Figure 17: Implementation of `File::EoF()`

Bonus Option 1: Design of an extension to the basic file system to allow for files that are up to 64kB long

To extend the current file system to support files of up to 64 KB (instead of a single 512-byte block), the inode structure and the mechanisms for file allocation, reading, and writing must be enhanced accordingly.

Inode Structure Changes

The original inode contains only a single direct pointer, allowing each file to occupy exactly one data block. To enable multi-block files, this field is replaced with an `indirect_block` pointer. The indirect block itself occupies one disk block and contains 128 integer entries (4 bytes each), with each entry storing the block number of a data block used by the file. This enables a file to grow to:

$$128 \text{ blocks} \times 512 \text{ bytes/block} = 64\,\text{KB}.$$

The modification can be summarized as:

- **Old field:** `int block_number;`
- **New field:** `int indirect_block;`

File Allocation

During file creation:

- A free block is allocated to serve as the file's `indirect_block`.
- All 128 entries in this block are initialized to `-1`, indicating that no data blocks have been assigned yet.

During a write operation:

- The logical block index is computed from the `current_position`.
- If the corresponding entry in the indirect block is unassigned (`-1`), a new data block is allocated and recorded in the indirect block.

Read/Write Changes

The `Read()` and `Write()` methods in `File` are updated to:

- Map the current file offset to the correct logical block.
- Load from or write to the appropriate physical data block referenced in the indirect block.
- Correctly process reads and writes that span multiple 512-byte blocks.

Function Modifications Summary

- **Inode:** Introduce and manage the `indirect_block` pointer.
- **FileSystem::CreateFile():** Allocate and initialize the indirect block.
- **FileSystem::DeleteFile():** Free all data blocks referenced in the indirect block, as well as the indirect block itself.
- **File::Read()/Write():** Use the indirect block to identify data blocks, compute logical offsets, and support multi-block operations.

This design enables file sizes of up to 64 KB while maintaining backward compatibility with the original file system. The single-level indirection strategy keeps the implementation simple, efficient, and easy to integrate with the existing structures.

Bonus Option 2: Implementation of the extensions proposed in Option 1

To enable file sizes of up to 64 KB, the single direct block pointer in each inode was replaced with an indirect block pointer. The following sections summarize the specific modifications made across the relevant classes and source files.

file_system.C: CreateFile()
In the extended implementation, the `CreateFile()` method allocates a dedicated `indirect_block` for each new file. A free block is obtained via `GetFreeBlock()`, initialized with 128 entries set to `-1`, and then written to disk. This indirect block serves as the table of data-block pointers for the file, enabling multi-block file growth.

```
bool FileSystem::CreateFile(int _file_id) {
    Console::puts("creating file with id:");
    Console::puti(_file_id);
    Console::puts("\n");

    // 1. Check if file already exists
    if (LookupFile(_file_id) != nullptr) {
        Console::puts("File already exists!\n");
        return false;
    }
    // 2. Find a free inode
    int inode_index = GetFreeInode();
    if (inode_index == -1) {
        Console::puts("No free inodes!\n");
        return false;
    }
    // 3. Find a free block for the indirect block
    int indirect_block_index = GetFreeBlock();
    if (indirect_block_index == -1) {
        Console::puts("No free blocks for indirect block!\n");
        return false;
    }
    // 4. Mark indirect block as used
    free_blocks[indirect_block_index] = 1;
    disk->write(FREELIST_BLOCK, free_blocks);
    // 5. Initialize the indirect block with 128 pointers set to -1
    unsigned char indirect_block_data[SimpleDisk::BLOCK_SIZE];
    for (int j = 0; j < 128; ++j) {
        ((int*)indirect_block_data)[j] = -1;
    }
    disk->write(indirect_block_index, indirect_block_data);
    // 6. Fill in inode
    Inode& inode = inodes[inode_index];
    inode.id = _file_id;
    inode.length = 0;
    inode.indirect_block = indirect_block_index;
    inode.valid = true;
    inode.fs = this;
    // 7. Save inode to disk
    inode.SaveToDisk(inode_index);

    Console::puts("File created successfully.\n");
    return true;
}
```

Figure 18: Allocating and initializing indirect block in `CreateFile()`

file_system.C: DeleteFile()
In the extended design, the deletion routine iterates through all entries of the file's indirect block and frees each referenced data block. After all allocated data blocks have been released, the indirect block itself is also deallocated. This process ensures complete cleanup of all disk resources associated with the file and correctly updates the free-block bitmap.

```
bool FileSystem::DeleteFile(int _file_id) {
    Console::puts("deleting file with id:");
    Console::puti(_file_id);
    Console::puts("\n");
    /* First, check if the file exists. If not, throw an error.
       Then free all blocks that belong to the file and delete/invalidate
       (depending on your implementation of the inode list) the inode. */

    // 1. Find the inode
    for (int i = 0; i < MAX_INODES; ++i) {
        if (inodes[i].valid && inodes[i].id == _file_id) {
            // 2. Free all blocks used by the file
            // 2a. Read indirect block
            unsigned char indirect_data[SimpleDisk::BLOCK_SIZE];
            disk->read(inodes[i].indirect_block, indirect_data);
            // 2b. Free each allocated data block
            for (int j = 0; j < 128; ++j) {
                int data_block = ((int*)indirect_data)[j];
                if (data_block >= DATA_BLOCK_START && data_block < size) {
                    free_blocks[data_block] = 0;
                }
            }
            // 2c. Free the indirect block itself
            int indirect_block_num = inodes[i].indirect_block;
            if (indirect_block_num >= DATA_BLOCK_START && indirect_block_num < size) {
                free_blocks[indirect_block_num] = 0;
            }
            // 2d. Save updated freelist
            disk->write(FREELIST_BLOCK, free_blocks);

            // 3. Clear the inode
            inodes[i] = Inode(); // reset to default
            inodes[i].fs = this; // restore fs pointer
            inodes[i].SaveToDisk(i);

            Console::puts("File deleted successfully.\n");
            return true;
        }
    }
    Console::puts("File not found!\n");
    return false;
}
```

Figure 19: Freeing all data and indirect blocks in `DeleteFile()`

file.C: File::Write()

The enhanced `Write()` method now supports data writes that span multiple 512-byte blocks. For each portion of the write operation, it invokes `GetBlockNumber()` to either retrieve the physical block associated with the current logical block index or allocate a new one if necessary. The corresponding data is then copied into that block's buffer and written back to disk, enabling seamless multi-block file growth.

```
int File::Write(unsigned int _n, const char *_buf) {
    Console::puts("writing to file\n");

    int bytes_written = 0;
    while (bytes_written < (int)_n && current_position < 128 * SimpleDisk::BLOCK_SIZE) {
        int logical_block = current_position / SimpleDisk::BLOCK_SIZE;
        int offset = current_position % SimpleDisk::BLOCK_SIZE;

        int block_num = GetBlockNumber(logical_block, true);
        if (block_num == -1) break;

        unsigned char temp_block[SimpleDisk::BLOCK_SIZE];
        fs->read_block_from_disk(block_num, temp_block);

        temp_block[offset] = _buf[bytes_written];
        fs->write_block_to_disk(block_num, temp_block);

        current_position++;
        bytes_written++;
    }

    if (current_position > (int)inode->length) {
        inode->length = current_position;
    }
    Console::puts("Finished writing to file.\n");
    return bytes_written;
}
```

Figure 20: Writing across multiple blocks using indirect pointers in `Write()`

file.C: File::Read()

The extended `Read()` method mirrors the behavior of the multi-block write operation. It supports reading data that spans multiple 512-byte blocks by determining the appropriate logical block index for each segment of the request. The method loads the corresponding physical block from disk and copies the required bytes into the user-provided buffer, continuing this process until the requested amount of data has been read or the end of the file is reached.

```
int File::Read(unsigned int _n, char *_buf) {
    Console::puts("reading from file\n");

    int bytes_read = 0;
    while (bytes_read < (int)_n && !EoF()) {
        int logical_block = current_position / SimpleDisk::BLOCK_SIZE
        int offset = current_position % SimpleDisk::BLOCK_SIZE;

        int block_num = GetBlockNumber(logical_block, false);
        if (block_num == -1) break;

        unsigned char temp_block[SimpleDisk::BLOCK_SIZE];
        fs->read_block_from_disk(block_num, temp_block);

        _buf[bytes_read] = temp_block[offset];
        current_position++;
        bytes_read++;
    }
    Console::puts("Finished reading file.\n");
    return bytes_read;
}
```

Figure 21: Reading across multiple blocks using indirect pointers in `Read()`

file.C: File::GetBlockNumber()
The `GetBlockNumber()` helper function encapsulates the logic required to work with indirect addressing. It determines the logical block index based on the file's `current_position`, loads the associated `indirect_block`, and then retrieves the corresponding physical data block if it has already been allocated. If the entry is unassigned, the method allocates a new data block, updates the indirect block accordingly, and persists the change. This routine centralizes the indirection mechanism and is used by both `Read()` and `Write()` to support multi-block file operations.

```
int File::GetBlockNumber(int logical_block_index, bool allocate_if_missing) {
    // Load indirect block
    unsigned char indirect_data[SimpleDisk::BLOCK_SIZE];
    fs->read_block_from_disk(inode->indirect_block, indirect_data);
    int* table = (int*)indirect_data;

    int block_num = table[logical_block_index];

    // If block not allocated and allowed to allocate
    if (block_num == -1 && allocate_if_missing) {
        for (int i = FileSystem::DATA_BLOCK_START; i < fs->get_disk_size(); ++i) {
            if (fs->is_block_free(i)) {
                fs->mark_block_used(i);
                table[logical_block_index] = i;

                // Save updated indirect block
                fs->write_block_to_disk(inode->indirect_block, (unsigned char*)table);
                return i;
            }
        }
        return -1; // Disk full
    }

    return block_num;
}
```

Figure 22: Managing indirect pointers via `GetBlockNumber()`

inode.C: Inode Fields

In the extended implementation, the original `block_number` field in the inode is replaced by the `indirect_block` pointer. This modification enables each file to reference up to 128 data blocks through a single-level indirection scheme.

```
class Inode
{
    friend class FileSystem; // The inode is in an uncomfortable position between
    friend class File;       // File System and File. We give both full access
    // to the Inode.

private:
    long id; // File "name"

    /* You will need additional information in the inode, such as allocation
       information. */
    unsigned int length;       // File size in bytes
    int indirect_block;         // Block number containing up to 128 data block pointers
    bool valid;                // Is this inode in use?

    FileSystem* fs; // It may be handy to have a pointer to the File system.
    // For example when you need a new block or when you want
    // to load or save the inode list. (Depends on your
    // implementation.)

/* You may need a few additional functions to help read and store the
   inodes from and to disk. */
public:
    Inode() : id(-1), length(0), indirect_block(-1), valid(false), fs(nullptr) {}

    void SaveToDisk(int index);
    void LoadFromDisk(int index);
};
```

Figure 23: Updated inode structure with `indirect_block` field

Collectively, these changes allow the file system to support read and write operations spanning up to 64 KB per file. All other file-system functions—such as `Mount()`, `Format()`, and `LookupFile()`—operate without modification, ensuring full backward compatibility with the original design.

Testing

To verify the correctness and robustness of both the base and extended versions of the file system, two categories of tests were conducted:

1. Repeated creation, writing, reading, and deletion of small files.
2. A bonus test validating support for large files (up to 64 KB) using the indirect-block mechanism.

The system was compiled and executed using:

```
make clean && make
make run
```

All diagnostic output was generated using `Console::puts()`, ensuring that each major operation and state transition was clearly visible during execution.

Basic File System Tests

The primary test routine is implemented in `exercise_file_system()` within `kernel.C`. This routine is executed iteratively over 30 loops (unless the large-file test is enabled). Each iteration performs the following steps:

- Create two files (`id = 1` and `id = 2`).
- Open each file and write a 20-character string to it.

- Reopen both files and read back their contents.
- Validate that the retrieved data matches the expected pattern, which alternates based on iteration parity.
- Delete both files and verify that their inodes are cleared.

This test ensures that file metadata, cached block contents, and inode updates are correctly preserved across open, close, and deletion operations.

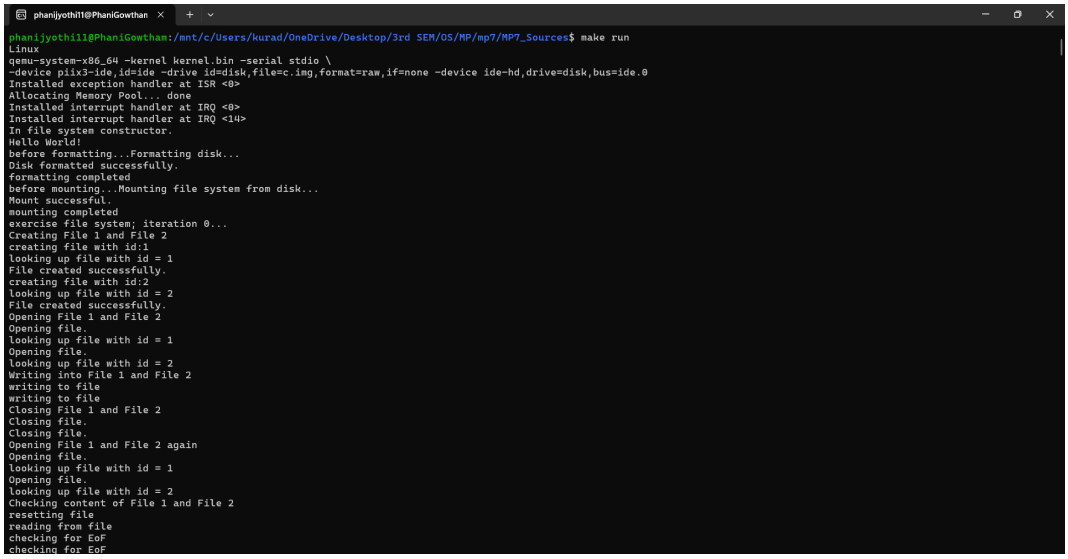


Figure 24: Main Case Output (Part 1): Basic File System

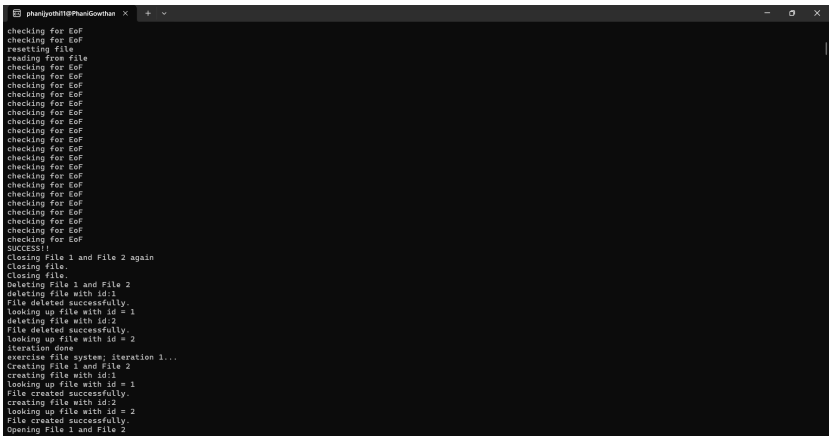


Figure 25: Main Case Output (Part 2): Basic File System (captured separately due to long output).

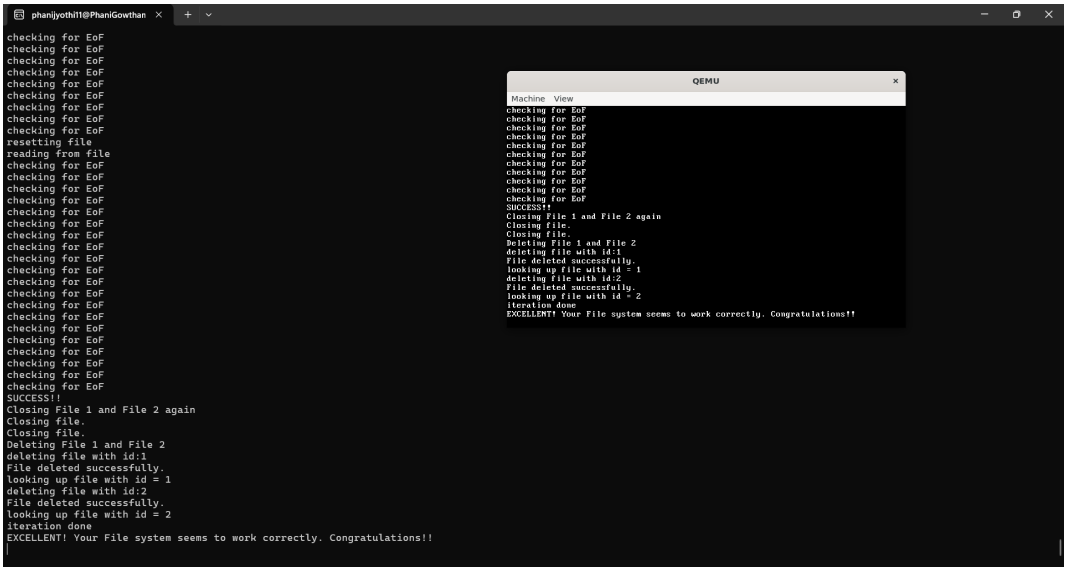


Figure 26: Main Case Output (Part 3): Basic File System (captured separately due to long output).

Conclusion: The 30-iteration test cycle completed without any assertion failures, confirming the correctness of file creation, block allocation, data I/O, and metadata management in the base file system.

Bonus Option 2: 64KB File Test

To evaluate the functionality introduced in Bonus Options 1 and 2, the following macro was enabled in `kernel.C`:

```
#define _USE_LARGE_FILE_TEST_
```

Enabling this directive bypasses the standard 30-iteration test and instead executes `test_large_file()`, which directly exercises the indirect-block mechanism.
The test proceeds as follows:

1. Create a file with identifier 42.
2. Populate a 64 KB buffer with a cyclic pattern of characters ('A' through 'Z').
3. Write the entire buffer to the file using multi-block operations.
4. Reset the file pointer and read the full contents into a second buffer.
5. Compare the two buffers byte-by-byte to ensure data integrity.
6. Delete the file and release all associated blocks.

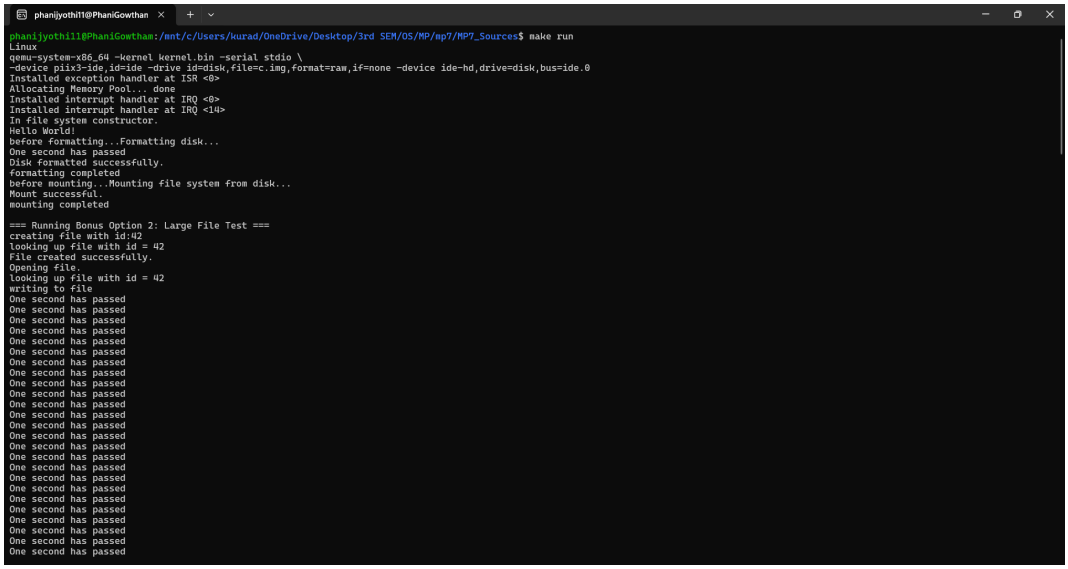


Figure 27: Code for `test_large_file()` defined in `kernel.C`

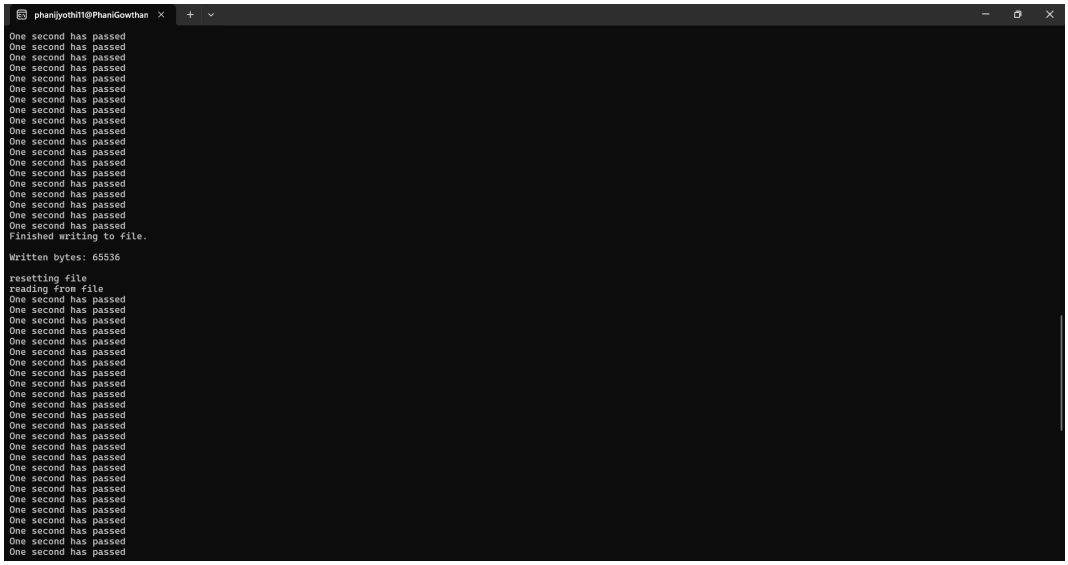


Figure 28: Test Output for Bonus Option 2

Figure 29: Test Output for Bonus Option 2 (captured separately due to long output)

Conclusion: The large-file test validates that the extended file system successfully implements single-level indirection. The results confirm that:

- File offsets are correctly mapped to their corresponding logical block indices.
- Physical data blocks are allocated, referenced, and accessed as expected.
- The system can write and read an entire 64 KB file spanning 128 contiguous blocks without error.

The test completes successfully with no assertion failures or data inconsistencies, demonstrating full support for extended file sizes.

Furthermore, the original small-file test (`exercise_file_system`) continues to pass without any modifications, confirming that the extended design remains fully backward-compatible and preserves correctness for both small and large files.